

ARCHITECTURE DECISION RECORD — TECHNOLOGY STACK

Flutter (Mobile) — chosen over React Native

Flutter compiles to native ARM code via its own rendering engine, avoiding the JS bridge latency React Native incurs. A single Dart codebase ships to both iOS and Android, halving mobile delivery effort. The declarative UI model is familiar to the team and acceptable as a contained Dart surface alongside our TS stack.

Next.js 14 (Web) — chosen over Vite SPA

The web platform requires SEO and fast initial loads. Next.js provides SSR and SSG out of the box without additional infrastructure. The App Router's Server Components reduce client JS bundle size, and built-in API Routes serve lightweight BFF needs without standing up a separate web server.

NestJS (Backend API) — chosen over Express / Fastify

NestJS enforces a modular, injectable architecture in TypeScript from day one. Guards, pipes, interceptors, and DI are built in, we do not need to construct this discipline ourselves. Its first-class integrations with TypeORM, Passport, and class-validator cover our core backend requirements with minimal extra config.

TypeORM (ORM) — chosen over Prisma

TypeORM's entity-decorator pattern mirrors NestJS conventions, keeping everything in TypeScript with no secondary schema language. Migration generation is CLI-driven and integrates cleanly into CI/CD. Prisma's ergonomics are good but its schema-first approach adds a context switch that TypeORM avoids.

PostgreSQL (Database) — chosen over MySQL / MongoDB

Our domain requires foreign keys, constraints, and JOIN-heavy queries — areas where a document database demands costly application-layer workarounds. PostgreSQL delivers full ACID compliance, rich indexing, and JSONB for any semi-structured payloads. Managed hosting (RDS, Cloud SQL) handles ops concerns around backups and replication.